Übungsaufgaben: Turingmaschinen und Entscheidbarkeit

Algorithmen und Komplexität

Aufgabe 1: Alphabete, Wörter und Sprachen

Sei

$$\Sigma = \{0, 1\}.$$

(a) Geben Sie die Mengen

$$\Sigma^0, \qquad \Sigma^1, \qquad \Sigma^2$$

explizit an.

(b) Was ist der Unterschied zwischen Σ^* und Σ^+?

(c) Welche der folgenden Mengen sind Sprachen über Σ?

$$L_1 = \{0, 01, 101\},$$

$$L_2 = \{w \in \Sigma^* \mid |w| \text{ ist gerade}\},$$

$$L_3 = \{0, 1, 2\}.$$

(d) Entscheiden Sie für jede der folgenden Mengen, ob sie endlich oder unendlich ist:

$$\Sigma, \qquad \Sigma^3, \qquad \Sigma^*, \qquad \{0^n 1^n \mid n \geq 0\}.$$

Lösung zu Aufgabe 1

(a)

$$\Sigma^0 = \{\varepsilon\}, \qquad \Sigma^1 = \{0, 1\},$$

und

$$\Sigma^2 = \{00, 01, 10, 11\}.$$

(b) Σ^* enthält alle endlichen Wörter über Σ, einschließlich des leeren Wortes ε:

$$\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i.$$

Dagegen enthält

$$\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i$$

nur die nichtleeren Wörter. Somit gilt

$$\Sigma^* = \Sigma^+ \cup \{\varepsilon\}.$$

(c) L_1 und L_2 sind Sprachen über Σ, da alle ihre Elemente Wörter über $\{0, 1\}$ sind. L_3 ist keine Sprache über Σ, da das Symbol 2 nicht zum Alphabet Σ gehört.

(d) Σ und Σ^3 sind endlich. Insbesondere gilt

$$|\Sigma^3| = 2^3 = 8.$$

Dagegen sind Σ^* und

$$\{0^n 1^n \mid n \geq 0\}$$

unendlich.

Aufgabe 2: Eine Turingmaschine ausführen

Wir betrachten die Sprache

$$L = \{10^k 1 \mid k \geq 0\}.$$

Eine Turingmaschine M verwendet die Zustände

$$q_0, q_1, q_2, q_{\text{accept}}, q_{\text{reject}}$$

und besitzt die folgenden Übergänge:

$$\delta(q_0, 1) = (q_1, 1, R),$$
$$\delta(q_0, 0) = (q_{\text{reject}}, 0, R), \qquad \delta(q_0, \sqcup) = (q_{\text{reject}}, \sqcup, R),$$
$$\delta(q_1, 0) = (q_1, 0, R),$$
$$\delta(q_1, 1) = (q_2, 1, R),$$
$$\delta(q_1, \sqcup) = (q_{\text{reject}}, \sqcup, R),$$

sowie

$$\delta(q_2, \sqcup) = (q_{\text{accept}}, \sqcup, R),$$
$$\delta(q_2, 0) = (q_{\text{reject}}, 0, R), \qquad \delta(q_2, 1) = (q_{\text{reject}}, 1, R).$$

(a) Führen Sie M auf dem Wort

$$w = 1001$$

Schritt für Schritt aus.

(b) Führen Sie M auf

$$w = 101$$

aus.

(c) Was geschieht bei den Eingaben

$$001, \qquad 100, \qquad 1101?$$

(d) Begründen Sie, warum M genau die Sprache L entscheidet.

Lösung zu Aufgabe 2

Wir markieren die aktuelle Kopfposition, indem wir den Zustand unmittelbar vor das gelesene Symbol schreiben.

(a) Für 1001 erhalten wir

$$q_0\,1001 \vdash 1q_1\,001 \vdash 10q_1\,01 \vdash 100q_1\,1 \vdash 1001q_2\,\sqcup \vdash 1001\,\sqcup\, q_{\text{accept}}.$$

Die Eingabe wird also akzeptiert.

(b) Für 101 gilt entsprechend

$$q_0\,101 \vdash 1q_1\,01 \vdash 10q_1\,1 \vdash 101q_2\,\sqcup \vdash 101\,\sqcup\, q_{\text{accept}}.$$

Also wird auch 101 akzeptiert.

(c) Das Wort 001 wird bereits verworfen, weil M im Startzustand q_0 eine 0 liest.

Beim Wort 100 erreicht M nach dem Lesen der beiden Nullen ein Blanksymbol im Zustand q_1. Daher wird verworfen.

Beim Wort 1101 wird die zweite 1 als abschließende 1 interpretiert. Danach befindet sich M im Zustand q_2, liest jedoch noch eine 0. Daher wird ebenfalls verworfen.

(d) Im Startzustand überprüft M, ob das erste Symbol eine 1 ist.

Anschließend liest M in q_1 beliebig viele Nullen:

$$0^k, \qquad k \geq 0.$$

Sobald die nächste 1 gelesen wird, wechselt M nach q_2.

In q_2 akzeptiert die Maschine genau dann, wenn unmittelbar danach das Blanksymbol folgt. Somit darf hinter dieser zweiten 1 kein weiteres Eingabesymbol stehen.

Daher akzeptiert M genau die Wörter der Form

$$10^k1.$$

Aufgabe 3: Eine Turingmaschine konstruieren

Betrachten Sie die Sprache

$$L = \{a^n b^n \mid n \geq 0\}.$$

Konstruieren Sie eine Turingmaschine, die L entscheidet.

Dabei müssen Sie nicht für jeden Übergang $\delta(q, a)$ einen vollständigen Zustandsgraphen angeben. Beschreiben Sie stattdessen präzise die Arbeitsweise der Maschine.

(a) Geben Sie die Idee des Algorithmus an.

(b) Beschreiben Sie, welche Bandsymbole zusätzlich zu a und b hilfreich sind.

(c) Führen Sie Ihre Maschine schematisch auf

$$aabb$$

aus.

(d) Begründen Sie die Korrektheit Ihrer Konstruktion.

(e) Begründen Sie, warum Ihre Maschine auf jeder Eingabe hält.

Lösung zu Aufgabe 3

Eine mögliche Maschine arbeitet nach dem Prinzip des schrittweisen *Abhakens* zusammengehöriger Zeichen.

Wir verwenden beispielsweise die zusätzlichen Bandsymbole

$$X \qquad \text{und} \qquad Y.$$

Dabei markiert X ein bereits verarbeitetes a und Y ein bereits verarbeitetes b.

(a) Die Maschine wiederholt folgende Schritte:

1. Suche von links nach rechts das erste noch nicht markierte a.
2. Ersetze dieses a durch X.
3. Laufe nach rechts und suche das erste noch nicht markierte b.
4. Existiert kein solches b, so verwirf.
5. Ersetze dieses b durch Y.
6. Laufe zum linken Rand der Eingabe zurück.

Gibt es kein unmarkiertes a mehr, so überprüft die Maschine, ob noch ein unmarkiertes b vorhanden ist. Falls ja, verwirft sie; andernfalls akzeptiert sie.

Zusätzlich muss überprüft werden, dass kein a rechts von einem bereits begonnenen b-Block vorkommt. Andernfalls würde beispielsweise

$$abab$$

fälschlicherweise akzeptiert.

(b) Die Markierungen

$$a \longmapsto X, \qquad b \longmapsto Y$$

erlauben es der Maschine, bereits bearbeitete Symbole von noch nicht bearbeiteten Symbolen zu unterscheiden.

(c) Für

$$aabb$$

kann der wesentliche Bandinhalt wie folgt aussehen:

$$aabb \longrightarrow Xabb \longrightarrow XaYb \longrightarrow XXYb \longrightarrow XXYY.$$

Danach gibt es weder ein unmarkiertes a noch ein unmarkiertes b. Somit akzeptiert die Maschine.

(d) Jedes markierte a wird genau einem markierten b zugeordnet.

Akzeptiert die Maschine, dann wurden alle a und alle b paarweise zugeordnet. Daher gibt es gleich viele a wie b.

Außerdem überprüft die Maschine, dass alle a vor allen b stehen. Somit besitzt jedes akzeptierte Wort die Form

$$a^n b^n.$$

Umgekehrt kann bei einem Wort $a^n b^n$ in jeder Runde genau ein a mit einem b gepaart werden. Nach n Runden sind alle Symbole markiert und die Maschine akzeptiert.

(e) In jeder erfolgreichen Runde wird mindestens ein bisher unmarkiertes a markiert. Da die Eingabe nur endlich viele Symbole besitzt, können nur endlich viele solcher Runden stattfinden.

Auch jeder einzelne Lauf über das Band benötigt nur endlich viele Schritte.

Daher hält die Maschine auf jeder Eingabe.

Aufgabe 4: Ein Band oder mehrere Bänder?

Eine k-Band-Turingmaschine besitzt k Bänder mit jeweils einem eigenen Lese-/Schreibkopf.

(a) Erklären Sie, warum eine Zwei-Band-Turingmaschine auf den ersten Blick mächtiger erscheint als eine gewöhnliche Ein-Band-Turingmaschine.

(b) Wie kann man die Inhalte mehrerer Bänder auf einem einzigen Band codieren?

(c) Wie kann die Ein-Band-Turingmaschine zusätzlich speichern, an welcher Position sich die jeweiligen Köpfe befinden?

(d) Erklären Sie, warum daraus folgt, dass Mehrband-Turingmaschinen nicht mehr Sprachen entscheiden können als Ein-Band-Turingmaschinen.

Lösung zu Aufgabe 4

(a) Eine Mehrbandmaschine kann gleichzeitig Informationen auf verschiedenen Bändern speichern und besitzt für jedes Band einen eigenen Kopf. Viele Algorithmen lassen sich dadurch wesentlich einfacher und häufig auch schneller formulieren.

(b) Die Inhalte der einzelnen Bänder können auf einem Band hintereinander gespeichert und durch ein spezielles Trennsymbol, etwa #, getrennt werden:

$$\# \, w_1 \, \# \, w_2 \, \# \, \cdots \, \# \, w_k \, \#.$$

(c) Man kann das Symbol, auf dem sich ein simulierter Kopf befindet, durch eine markierte Variante des entsprechenden Bandsymbols darstellen.

Ist beispielsweise

$$a$$

ein Bandsymbol, so könnte

$$\dot{a}$$

bedeuten, dass der simulierte Kopf gerade auf dieser Position steht.

(d) Die Ein-Band-Maschine kann einen Schritt der Mehrbandmaschine simulieren, indem sie zunächst das gesamte codierte Band durchsucht und die aktuell gelesenen Symbole aller Köpfe bestimmt.

Anschließend führt sie die entsprechenden Schreiboperationen durch und verschiebt die Markierungen der simulierten Köpfe.

Ein einzelner Schritt der Mehrbandmaschine kann dadurch zwar viele Schritte der Ein-Band-Maschine benötigen. Entscheidend ist hier jedoch nicht die Laufzeit, sondern die Berechenbarkeit.

Somit kann jede Mehrband-Turingmaschine durch eine Ein-Band-Turingmaschine simuliert werden.

Mehrere Bänder verändern also den Komfort beziehungsweise die Effizienz, aber nicht die Menge der berechenbaren beziehungsweise entscheidbaren Probleme.

Aufgabe 5: Nichtdeterministische Turingmaschinen

Bei einer nichtdeterministischen Turingmaschine kann es für eine Kombination aus Zustand und gelesenem Bandsymbol mehrere mögliche Übergänge geben.

(a) Was bedeutet es, dass eine nichtdeterministische Turingmaschine eine Eingabe w akzeptiert?

(b) Betrachten Sie die Sprache

$$L = \{w \in \{0,1\}^* \mid w \text{ enthält } 001 \text{ als Teilwort}\}.$$

Beschreiben Sie eine nichtdeterministische Turingmaschine für L, die zunächst eine Position des Wortes "rät".

(c) Was geschieht bei der Eingabe

$$w = 10010?$$

(d) Muss jede Berechnung der Maschine akzeptieren, damit w akzeptiert wird?

(e) Warum folgt aus Nichtdeterminismus nicht, dass grundsätzlich mehr Sprachen entscheidbar werden?

Lösung zu Aufgabe 5

(a) Eine nichtdeterministische Turingmaschine besitzt für eine Eingabe im Allgemeinen einen ganzen Berechnungsbaum.

Die Eingabe w wird akzeptiert, wenn es *mindestens einen* Berechnungspfad gibt, der einen akzeptierenden Zustand erreicht.

(b) Die Maschine kann beim Lesen des Wortes an jeder Position nichtdeterministisch zwischen zwei Möglichkeiten wählen:

1. weiter nach rechts gehen oder
2. vermuten, dass an der aktuellen Position das gesuchte Teilwort 001 beginnt.

Im zweiten Fall prüft die Maschine die nächsten drei Zeichen. Sie akzeptiert genau dann auf diesem Berechnungspfad, wenn diese

$$0, 0, 1$$

sind.

(c) Für

$$w = 10010$$

existiert ein Berechnungspfad, der die zweite Position als Anfang des Teilworts auswählt:

$$1\underline{001}0.$$

Dieser Pfad akzeptiert. Daher akzeptiert die gesamte nichtdeterministische Maschine die Eingabe.

(d) Nein. Es genügt ein einziger akzeptierender Berechnungspfad.

Andere Pfade dürfen verwerfen.

(e) Der gesamte Berechnungsbaum einer nichtdeterministischen Turingmaschine kann von einer deterministischen Turingmaschine systematisch simuliert werden.

Dabei werden die verschiedenen Berechnungspfade beispielsweise breitensuchartig betrachtet.

Für die in der Vorlesung betrachteten Entscheidungsprobleme verändert Nichtdeterminismus daher nicht die grundsätzliche Berechenbarkeit.

Aufgabe 6: Von Problemen zu Sprachen

Ein Entscheidungsproblem besitzt für jede zulässige Eingabe genau eine der Antworten

JA oder NEIN.

(a) Betrachten Sie das Problem

$$D_{\text{even}}: \quad \text{“Ist eine gegebene natürliche Zahl gerade?”}$$

Beschreiben Sie eine mögliche binäre Codierung der Eingaben.

(b) Formulieren Sie die zugehörige Sprache

$$L_{\text{even}} \subseteq \{0,1\}^*.$$

(c) Betrachten Sie nun das Entscheidungsproblem

$$D_{\text{path}}: \quad \text{“Gibt es in einem gegebenen Graphen } G \text{ einen Weg von } s \text{ nach } t\text{?”}$$

Warum kann auch dieses Problem als Sprache über $\{0,1\}$ aufgefasst werden?

(d) Warum ist die konkrete Codierung für die grundsätzliche Frage der Entscheidbarkeit normalerweise nicht wesentlich?

Lösung zu Aufgabe 6

(a) Eine natürliche Zahl kann durch ihre Binärdarstellung codiert werden.

Beispielsweise wird

$$6$$

durch

$$110$$

dargestellt.

(b) Wir können definieren

$$L_{\text{even}} = \{w \in \{0,1\}^* \mid w \text{ codiert eine gerade natürliche Zahl}\}.$$

Bei der üblichen Binärdarstellung ist eine positive Zahl genau dann gerade, wenn ihr letztes Bit 0 ist.

(c) Ein endlicher Graph kann durch einen endlichen Binärstring beschrieben werden. Beispielsweise können die Anzahl der Knoten, die Adjazenzmatrix oder eine Liste der Kanten sowie die beiden ausgezeichneten Knoten s und t codiert werden.

Damit erhält jede Probleminstanz einen endlichen String

$$\langle G, s, t\rangle \in \{0,1\}^*.$$

Die Sprache des Entscheidungsproblems ist dann

$$L_{\text{path}} = \{\langle G, s, t\rangle \mid \text{in } G \text{ existiert ein Weg von } s \text{ nach } t\}.$$

(d) Verschiedene vernünftige endliche Codierungen lassen sich algorithmisch ineinander übersetzen.

Die konkrete Darstellung kann daher die Laufzeit beeinflussen, aber sie ändert nicht grundsätzlich, ob das Problem überhaupt algorithmisch entscheidbar ist.

Aufgabe 7: Erkennbar oder entscheidbar?

Sei $L \subseteq \Sigma^*$ eine Sprache.

(a) Erklären Sie den Unterschied zwischen einer Turingmaschine, die L *erkennt*, und einer Turingmaschine, die L *entscheidet*.

(b) Was darf ein Erkenner für L auf einer Eingabe

$$w \notin L$$

tun?

(c) Was muss ein Entscheider auf einer solchen Eingabe tun?

(d) Zeigen Sie: Jede entscheidbare Sprache ist auch erkennbar.

(e) Warum gilt die umgekehrte Aussage nicht automatisch?

Lösung zu Aufgabe 7

(a) Eine Turingmaschine M erkennt L, wenn für alle Wörter $w \in L$ gilt:

$$M \text{ akzeptiert } w.$$

Für ein Wort $w \notin L$ darf M entweder verwerfen oder unendlich lange weiterlaufen.

Eine Turingmaschine entscheidet L, wenn sie auf *jeder* Eingabe hält und

$$w \in L \iff M \text{ akzeptiert } w.$$

(b) Für $w \notin L$ darf ein Erkenner entweder verwerfen oder niemals halten.

(c) Ein Entscheider muss nach endlich vielen Schritten halten und verwerfen.

(d) Sei M ein Entscheider für L.

Dann hält M auf jeder Eingabe und akzeptiert genau die Wörter aus L. Insbesondere akzeptiert M jedes Wort aus L.

Somit erfüllt M automatisch auch die Anforderungen an einen Erkenner.

Daher gilt

$$L \text{ entscheidbar} \implies L \text{ erkennbar}.$$

(e) Bei einem Erkenner kann für

$$w \notin L$$

eine unendliche Berechnung auftreten.

Beobachtet man eine Maschine, die bereits sehr lange läuft, kann man im Allgemeinen nicht allein daraus schließen, ob sie irgendwann noch hält oder für immer weiterläuft.

Genau dieses Phänomen führt zum Halteproblem.

Aufgabe 8: Das Halteproblem

Das Halteproblem sei

$$\text{HALT} = \{\langle M, w\rangle \mid M \text{ hält bei Eingabe } w\}.$$

(a) Beschreiben Sie eine Turingmaschine, die HALT erkennt.

(b) Warum ist diese Maschine im Allgemeinen kein Entscheider für HALT?

(c) Ein Student schlägt folgenden Algorithmus vor:

> Simuliere M auf w für 10^{100} Schritte. Falls M bis dahin hält, antworte JA. Andernfalls antworte NEIN.

Erklären Sie den Fehler.

(d) Angenommen, es gäbe einen Entscheider H mit

$$H(\langle M, w\rangle) = \begin{cases} \text{JA}, & \text{falls } M \text{ auf } w \text{ hält,} \\ \text{NEIN}, & \text{sonst.} \end{cases}$$

Wir konstruieren daraus eine Maschine D, die auf Eingabe $\langle M\rangle$ folgendes tut:

1. Berechne
$$H(\langle M, \langle M\rangle\rangle).$$
2. Falls H mit JA antwortet, laufe für immer.
3. Falls H mit NEIN antwortet, halte.

Was geschieht, wenn wir D auf ihre eigene Beschreibung $\langle D\rangle$ anwenden?

(e) Welche Schlussfolgerung ergibt sich daraus?

Lösung zu Aufgabe 8

(a) Eine Maschine U kann auf Eingabe
$$\langle M, w\rangle$$
einfach die Berechnung von M auf w simulieren.
Sobald M hält, akzeptiert U.
Somit gilt:
$$\langle M, w\rangle \in \text{HALT} \quad \Longrightarrow \quad U \text{ akzeptiert.}$$
Also ist HALT erkennbar.

(b) Falls M auf w niemals hält, läuft auch die Simulation von U niemals zu Ende.
Die Maschine U kann daher in diesem Fall nicht notwendigerweise mit NEIN antworten.
Sie ist somit zunächst nur ein Erkenner.

(c) Eine Maschine kann beispielsweise erst nach

$$10^{100} + 1$$

Schritten halten.

Aus der Tatsache, dass sie nach 10^{100} Schritten noch nicht gehalten hat, folgt daher nicht, dass sie niemals halten wird.

Kein fester endlicher Zeithorizont kann dieses Problem lösen.

(d) Betrachten wir

$$D(\langle D \rangle).$$

Es gibt zwei Möglichkeiten.

Angenommen zunächst,

$$H(\langle D, \langle D \rangle \rangle) = \textsc{Ja}.$$

Dann behauptet H, dass D auf $\langle D \rangle$ hält.

Nach Definition von D führt die Antwort Ja jedoch dazu, dass D für immer läuft.

Widerspruch.

Angenommen nun,

$$H(\langle D, \langle D \rangle \rangle) = \textsc{Nein}.$$

Dann behauptet H, dass D auf $\langle D \rangle$ nicht hält.

Nach Definition von D hält D in diesem Fall aber.

Wieder erhalten wir einen Widerspruch.

In kompakter Form:

$$D(\langle D \rangle) \text{ hält } \iff D(\langle D \rangle) \text{ hält nicht.}$$

(e) Unsere einzige zusätzliche Annahme war die Existenz eines Entscheiders H für HALT.

Diese Annahme führt zu einem Widerspruch.

Daher kann ein solcher Entscheider nicht existieren. Also ist

$$\boxed{\text{HALT nicht entscheidbar.}}$$

Gleichzeitig haben wir in Teil (a) gesehen, dass HALT erkennbar ist.

Das Halteproblem liefert somit ein konkretes Beispiel für eine Sprache, die erkennbar, aber nicht entscheidbar ist.

Zusatzaufgabe: Begriffe wirklich verstanden?

Entscheiden Sie, ob die folgenden Aussagen wahr oder falsch sind. Begründen Sie jeweils kurz.

(a) Jede Turingmaschine hält auf jeder Eingabe.

(b) Jede entscheidbare Sprache ist erkennbar.

(c) Jede erkennbare Sprache ist entscheidbar.

(d) Eine nichtdeterministische Turingmaschine akzeptiert nur dann, wenn alle ihre Berechnungspfade akzeptieren.

(e) Eine Zwei-Band-Turingmaschine kann grundsätzlich mehr Sprachen entscheiden als eine Ein-Band-Turingmaschine.

(f) Für ein unentscheidbares Problem gibt es überhaupt keinen Algorithmus, der auf irgendeiner Eingabe eine korrekte Antwort liefern kann.

(g) Das Halteproblem ist erkennbar.

(h) Das Halteproblem ist entscheidbar.

Lösung zur Zusatzaufgabe

(a) Falsch. Eine Turingmaschine kann unendlich lange laufen.

(b) Wahr. Ein Entscheider ist insbesondere ein Erkenner.

(c) Falsch. Das Halteproblem ist ein Gegenbeispiel.

(d) Falsch. Ein akzeptierender Berechnungspfad genügt.

(e) Falsch. Mehrband-Turingmaschinen können durch Ein-Band-Turingmaschinen simuliert werden.

(f) Falsch. Unentscheidbarkeit bedeutet, dass es keinen Algorithmus gibt, der *für jede Eingabe* nach endlich vielen Schritten die korrekte Ja-/Nein-Antwort liefert.

(g) Wahr. Man simuliert die gegebene Maschine und akzeptiert, sobald sie hält.

(h) Falsch.